Scalable Join Discovery with LLMs and Recall Guarantees

Saeed Fathollahzadeh
University of Waterloo — Data Intelligence Lab

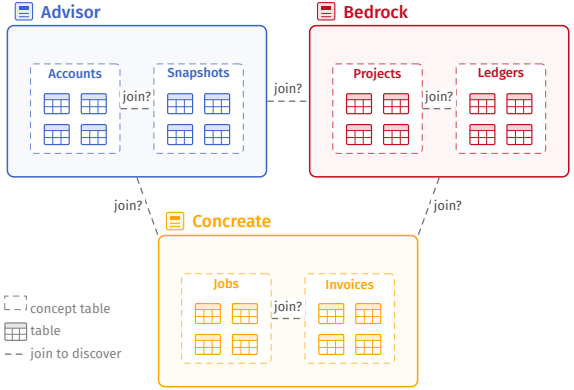
Context | Enterprise Data Needs a Join Graph

The setting

- Many businesses, each with **50 to 8K tables**
- Staging copies, snapshots, ERP exports
- Foreign keys are **rarely declared**

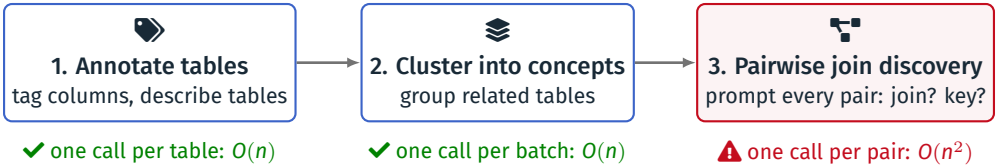
Before joining: concept tables

- Tag the columns of every table
- Build **concept tables**: cluster similar or overlapping tables
- Discover joins between concepts, **inside a business** and **across businesses**



Join graph = which tables join, **on which key, in which direction**

Context | How Join Graphs Are Built Today — Three LLM Steps



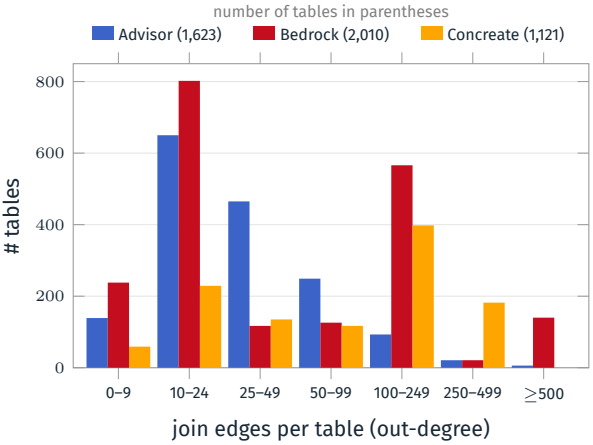
What works

- The LLM reads names, descriptions, and values together
- It recognizes business entities that statistics miss

What breaks

- Step 3 dominates cost and time
- Its output graph is **dense**, **redundant**, and hard to trust

Evidence | LLM-built Join Graphs Are Dense



- **Bedrock**: **36%** of tables have ≥**100** join edges; **140** tables have ≥500
- **Concreate**: median table has **121** join edges
- Max: one table joins **1,450** others

Real schemas have a handful of foreign keys per table — not hundreds.

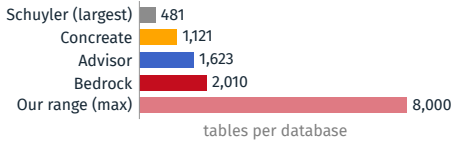
Source: exported join graphs of three businesses (top 5,000 nodes each).

Problem 1 | Clustering Tables into Concepts Does Not Scale

Way 1: the LLM builds concept tables

- All tables must fit in **one prompt**, or be split into batches that each see **part of the schema**
- Batches name and split the same concept differently

Evidence: in Schuyler's study[2], GPT-4o **omitted tables** and **hallucinated** new ones on the largest databases (up to 481 tables).

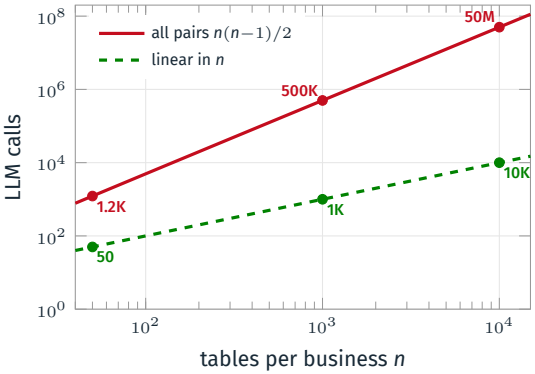


Way 2: embed and cluster (Schuyler[2])

- Fine-tuned table embeddings + clustering, **no labels**
- Evaluated on **29–481 tables**; #clusters searched in **[2, 20)**
- **Disjoint** clusters, yet a table can belong to several concepts

Clusters decide which pairs are compared. In Schuyler's benchmark[2], only **60%** of foreign keys stay inside one cluster: join only **within** clusters ⇒ lose up to **40%** of joins; join **across** clusters ⇒ back to all pairs.

Problem 2 | Pairwise Prompting Costs Grow Quadratically



- Every call carries **two schemas** and **sample rows**
- 10K tables ⇒ ≈**50M** calls per business
- Almost every call answers **"no join"**

Evidence: Advisor, Bedrock, and Concreate contain 0.6M–2.0M table pairs in the exported subgraph alone.

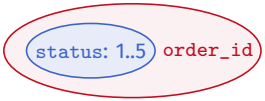
Goal: LLM calls that grow with **what joins**, not with **every pair**.

Problem 3 | Cheap Signals Produce Wrong Joins — and Miss Real Ones

False joins from accidental overlap

- Surrogate IDs $1..m$ are **always** contained in $1..n$, $n \geq m$
- Status codes, years, and flags overlap everywhere
- Street names overlap last names ("Lincoln", "Washington")

Evidence: LakeBench [1]: JOSIE [3] ranks the street-name column in the top-k for a last-name query.



$1..10^6$ contains 1..5, but **contained** $\neq$ **join**

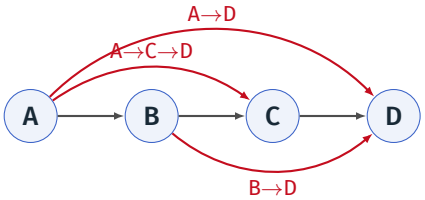
Real joins that look imperfect

- Staging copies keep only recent rows $\Rightarrow$ **partial** containment
- Formats differ (`00123` vs. `123`, `P-123`) $\Rightarrow$ **zero** overlap
- One key, many names: `PROJECT_CODE`, `PRJ_ID`, `ENGAGEMENT`

Evidence: LakeBench [1]: on fuzzy-overlap joins, JOSIE [3] reaches **0%** top-10 accuracy, DeepJoin [5] **33%**.

A join needs **meaning** and **overlap**. Statistics see overlap; only the LLM sees meaning.

Problem 4 | Redundant Joins Blow Up the Graph



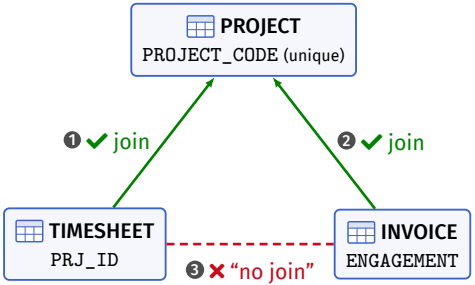
— chain — shortcuts already implied by the chain

- A key shared by k tables yields $O(k^2)$ pairwise edges
- The answers **follow** from k references to one owner
- Queries over the graph must choose among many paths

Caution: joins do not always chain

`Employee`→`Dept`→`Site` is the *office* site, while `Employee`→`Site` may be the *home* site. A shortcut is redundant **only if the same key values flow along the path.**

Problem 5 | Independent Answers Are Inconsistent



one prompt per pair: three separate answers

- Prompts **1** and **2**: both columns refer to `PROJECT_CODE` → **join**
- Prompt **3** sees only `TIMESHEET` and `INVOICE`: names differ, a few sample rows do not overlap → **"no join"**
- Together they **contradict**: two columns with the same key **do** join through `PROJECT_CODE`
- The graph shows all three answers, but **not which one is wrong**

One ring instead of three pairs: `PROJECT_CODE` ← {`PRJ_ID`, `ENGAGEMENT`} — one decision, no contradiction.

Existing Work | Non-LLM, LLM, and Hybrid Join Discovery

	Non-LLM IND discovery[10], Auto-BI[6], LSH Ensemble[4], JOSIE[3], DeepJoin[5]	**LLM** all-pairs prompting	**Hybrid** LOTUS[7], BARGAIN[8], FDJ[9], Schuyler[2]	**Ours** key domains
Decides by meaning	✗ overlap & names	✓	✓ if routed to LLM	✓ LLM per ring
LLM calls grow with	n/a	✗ table pairs	✗ candidate pairs	✓ key domains
Recall guarantee	✗	n/a (asks all)	✓ per operator	✓ vs. all-pairs LLM
Needs labels	✗ (Auto-BI)	✓ none	✓ none	✓ own LLM answers
Consistent graph	partly (Auto-BI)	✗ independent	✗ independent	✓ star per key

Non-LLM: cheap, but blind to meaning
LakeBench[1]: no method wins everywhere

LLM: accurate, but $O(n^2)$
the oracle we want to match

Hybrid: cheaper per pair, still pairwise
BARGAIN[8] defers ≈80% of pairs[9]

[1] Y. Deng, C. Chai, L. Cao, et al. **LakeBench**: A Benchmark for Discovering Joinable and Unionable Tables in Data Lakes. *PVLDB* 17(8), 2024.

[2] L. Laskowski, F. Panse, M. Hladik, J. Portisch, F. Naumann. **Schuyler**: Self-Supervised Clustering of Tables in Relational Databases. *PVLDB* 19(4), 2025.

[3] E. Zhu, D. Deng, F. Nargesian, R. J. Miller. **JOSIE**: Overlap Set Similarity Search for Finding Joinable Tables in Data Lakes. *SIGMOD* 2019.

[4] E. Zhu, F. Nargesian, K. Q. Pu, R. J. Miller. **LSH Ensemble**: Internet-Scale Domain Search. *PVLDB* 9(12), 2016.

[5] Y. Dong, C. Xiao, T. Nozawa, M. Enomoto, M. Oyamada. **DeepJoin**: Joinable Table Discovery with Pre-trained Language Models. *PVLDB* 16(10), 2023.

[6] Y. Lin, Y. He, S. Chaudhuri. **Auto-BI**: Automatically Build BI-Models Leveraging Local Join Prediction and Global Schema Graph Optimization. *PVLDB* 16(10), 2023.

[7] L. Patel, S. Jha, C. Guestrin, M. Zaharia. **LOTUS**: Enabling Semantic Queries with LLMs Over Tables of Unstructured and Structured Data. *arXiv:2407.11418*, 2024.

[8] S. Zeighami, S. Shankar, A. Parameswaran. Cut Costs, Not Accuracy: LLM-Powered Data Processing with Guarantees (**BARGAIN**). *SIGMOD* 2026.

[9] S. Zeighami, S. Shankar, A. Parameswaran. **Featurized-Decomposition Join**: Low-Cost Semantic Joins with Guarantees. *arXiv:2512.05399*, 2025.

[10] F. Tschirschnitz, T. Papenbrock, F. Naumann. Detecting **Inclusion Dependencies** on Very Many Tables. *ACM TODS* 42(3), 2017.